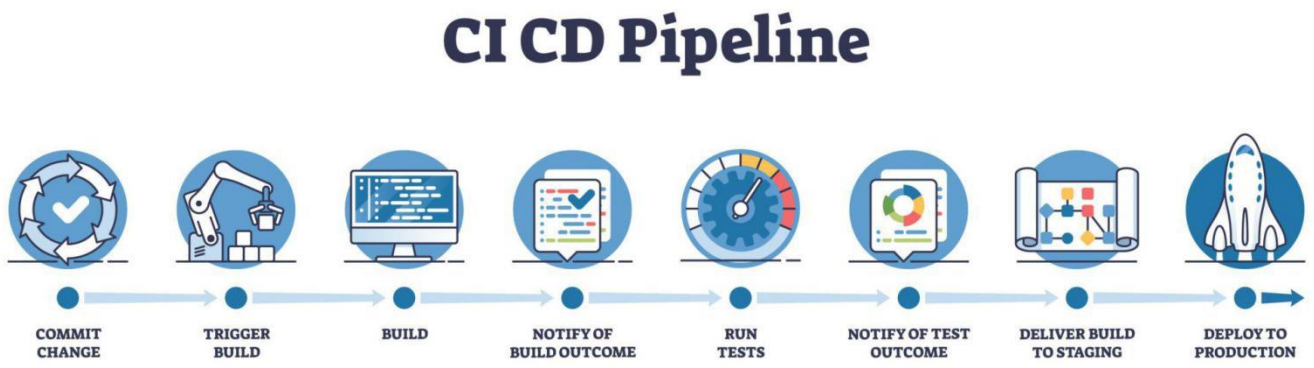
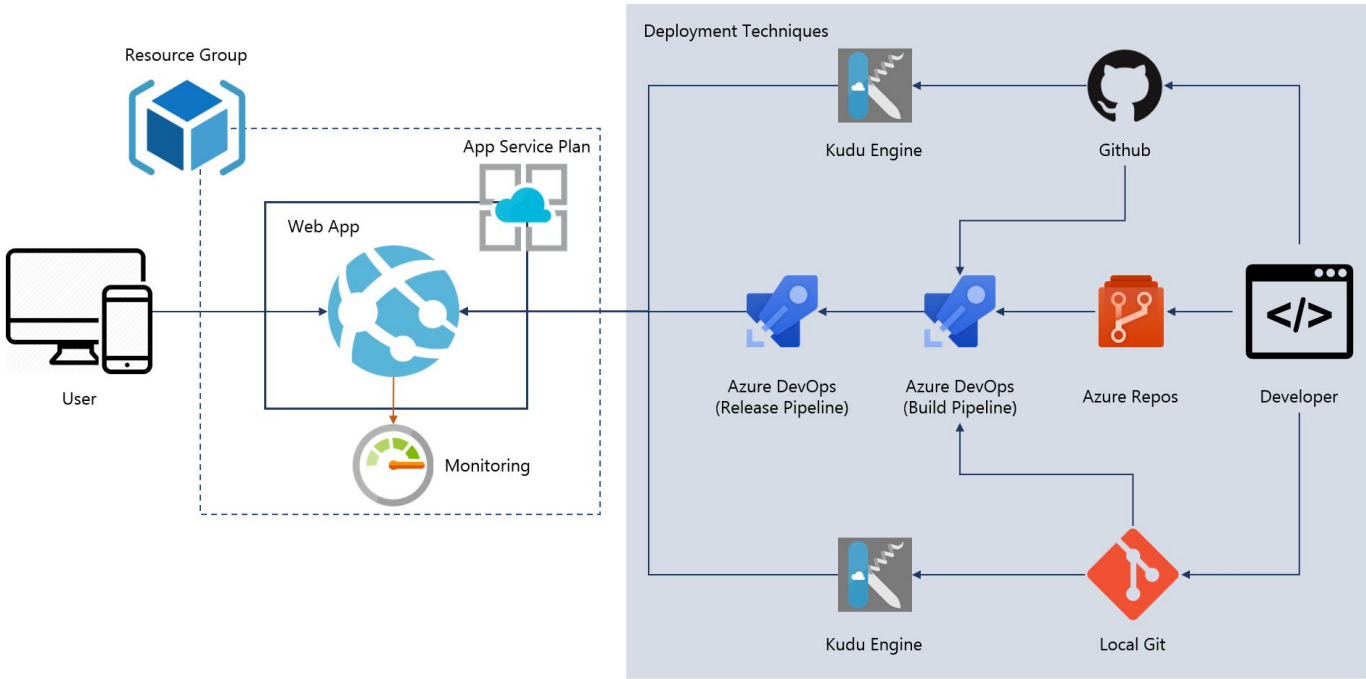
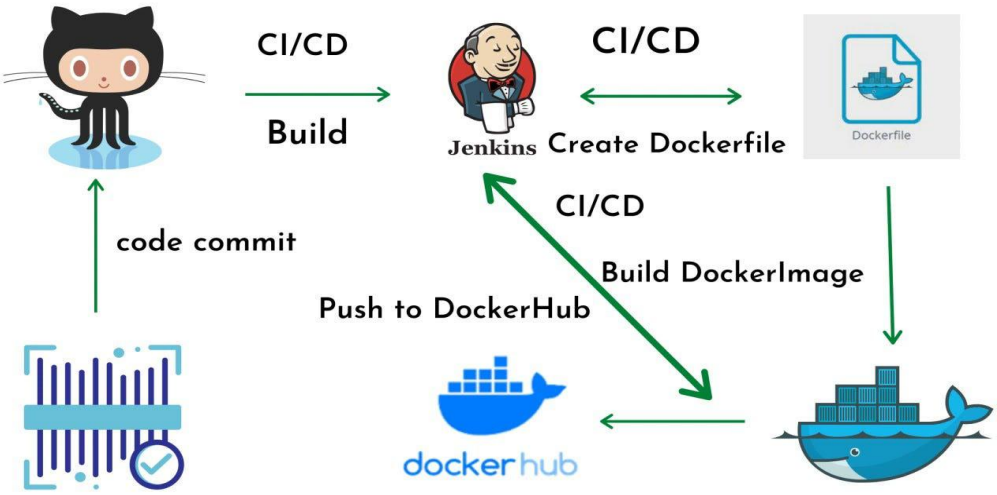


This below diagram illustrates the CI/CD Pipeline architecture for deploying a Web App on Microsoft Azure.



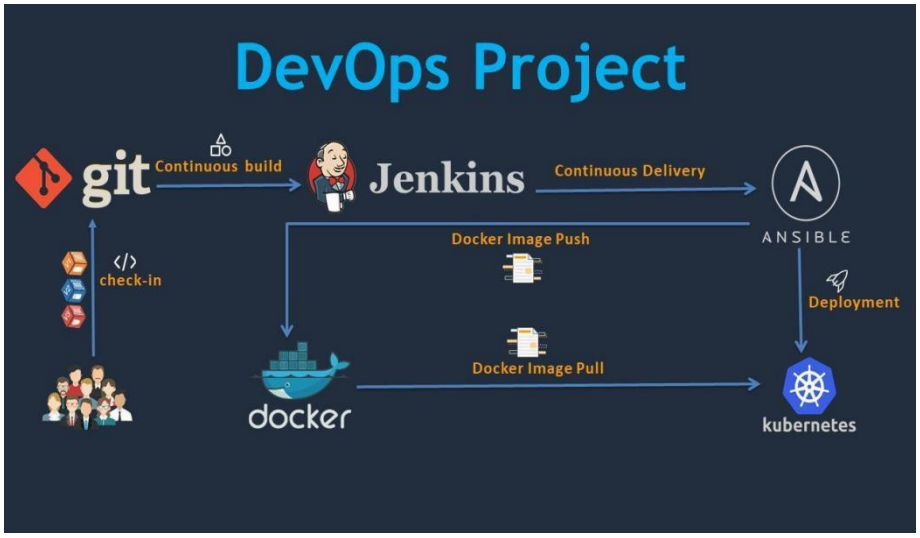
The below image displays a work-flow diagram for a Continuous Integration and Continuous Deployment (CI/CD) pipeline, illustrating how software changes are managed from development to deployment.

- Code Commit: Developer code is committed to a repository, represented by the GitHub Octocat.
- Automation Server: Jenkins automates the process by building the project and creating a Dockerfile.
- Containerization: A Docker image is built based on the Dockerfile.
- Deployment: The final Docker image is pushed to a registry like Docker Hub for distribution.

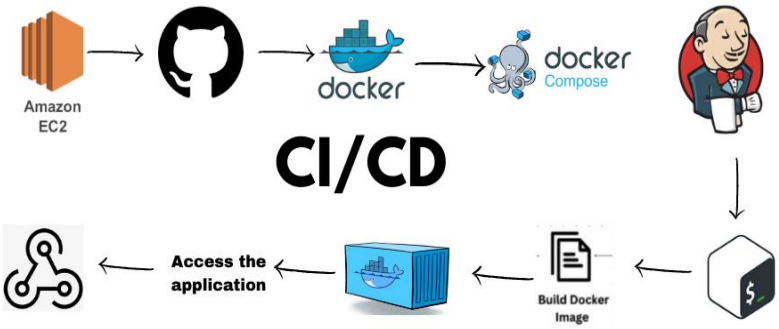


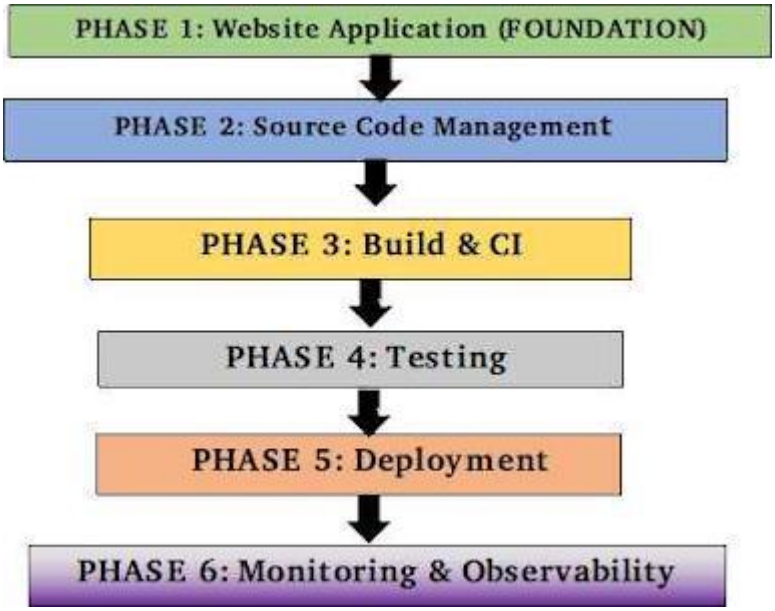
This image illustrates a DevOps Project work-flow showcasing a Continuous Integration/Continuous Delivery (CI/CD) pipeline.

- **Version Control & Trigger:** Developers check in code to Git, triggering automated build processes.
- **Automation Server:** Jenkins acts as the orchestrator for the pipeline, handling continuous integration and preparing for continuous delivery.
- **Containerization:** Docker is used to build container images, which are pushed to a registry and subsequently pulled for deployment.
- **Configuration Management & Orchestration:** Ansible automates configuration and deployment, while Kubernetes manages the containerized applications in production.



(or)





Improve website UI (DevOps theme)

- Create Dockerfile
- Jenkins pipeline (CI)
- Selenium test
- EC2 deployment
- Domain mapping
- Grafana dashboard
- Resume + GitHub polish

1 How a Professional DevOps Project UI Should FEEL :

Think of your website as:

“A living DevOps system, not a blog”

When someone (recruiter / senior engineer / friend) opens your site, they should immediately feel:

✔ This person understands DevOps life-cycle

✔ This project is real, running, monitored

✔ This is not a tutorial copy-paste

<u>Visual Personality :</u>

✓ Dark theme (tech + ops feel)

✓ Minimal colors (blue / green)

✓ Dashboard-style layout

✓ Clear sections (no clutter)

The Following is the Example Diagram of the Grafana Dashboard :

Pipelines list

The Pipelines list shows all the pipelines that you have permission to view. Access CloudBees CD/RO Pipeline list by navigating to Release Orchestration Pipeline Runs.

A. Access pipeline and pipeline run data.

- Modify pipeline description and tags by selecting the Kubernetesblue-info-h-icon.png[Information] information icon.
- Edit a pipeline by selecting the pipeline `name` link.
- Modify a pipeline run by selecting the `name` link for the **Run instance**.

B. Manage run instances.

- Access pipeline CloudBees CI build data by selecting the CloudBees CI view build data icon.
- Choose to proceed or abort a CloudBees CI pipeline task by selecting the CloudBees CI pipeline approval icon.
- Review pipeline run data in the **Project**, **Stage**, **Start time**, **Duration**, and **Restarted** columns.

C. Add or remove pipelines.

- Create a new or copy an existing pipeline by selecting New icon.
- Remove a pipeline.
  a. Select the pipeline to be deleted.
  b. Select the Delete Delete icon.
- Deselect chosen pipelines by selecting the Select None None icon.

D. Manage pipelines.

- Run the pipeline by selecting the Run icon.
- Select the Views icon to navigate to one of the following pipeline features.
  - Pipeline editor
  - Pipeline run
  - DSL editor
  - Pipeline portfolio
  - Pipeline portfolio list

The following is the Link for the Pipeline UI work-flow :
https://docs.cloudbees.com/docs/cloudbees-cd/latest/pipelines/pipeline-ui

Top Navigation (Always Visible)

Home | DevOps Work-flow | CI/CD Pipeline | Live Dashboard | Project Info | Contact

---

- ◆
- ◆
- ◆

➢ **Answer this question instantly:**

Purpose: Confidence
Content:

DevOps Life-cycle Implementation
Automated CI/CD • Cloud Deployment • Live Monitoring
[View Live Dashboard]
[View Pipeline]

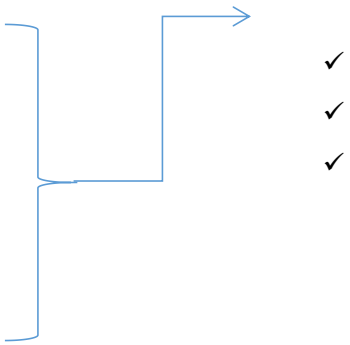
✔ Big title
✔ 2 buttons
✔ No scrolling needed yet

⚠

→ → →

Verdict :

Home Page = System health + Confidence -------> NOT documentation



This page is for **engineers**, not HR.
Show:

- ✓ Jenkins pipeline stages
- ✓ Build logs (last run)
- ✓ Docker image version
- ✓ Git commit hash

Example layout:

✔ Even screenshots are fine initially
✔ Later connect real Jenkins API

This is where you separate yourself.

? Should live dashboard be on Home Page?

Answer:

FULL dashboard ---- NO

Status Summary ---- YES

But the **Full Grafana dashboard** ⟶ **Separate Page**

- ✧ CPU usage
- ✧ Memory usage
- ✧ App health
- ✧ Request count
- ✧ Container status

Embed Grafana panel (iframe).

This Proves :

- ✧ System is running
- ✧ You know observability
- ✧ You didn’t fake project

- ✓
- ✓
- ✓
- ✓
- ✓

- ✓
- ✓
- ✓
- ✓

✅ FINAL WEBSITE TREE STRUCTURE (PROFESSIONAL)

Below is an **exact, production-style directory tree** for your **complete DevOps-themed website**, designed so that:

- ➢ UI is **clean & modular**
- ➢ Each section has a **clear responsibility**
- ➢ It smoothly transitions from **Node.js → Docker → CI/CD → AWS → Monitoring**
- ➢ You **do NOT mix logic, UI, and dashboards**

dashboards/

Phase	What part
Source	GitHub repo
Build	Dockerfile
CI	Jenkinsfile
Test	Selenium tests
Deploy	EC2 container
Monitor	Grafana / Prometheus
UI	public/